

Unit III

SYSTEM ARCHITECTURE

System Architecture

- Deciding software components, their interaction, and their placement leads to an instance of a software architecture, also known as a **system architecture**.
- **Centralized organizations**
- **Decentralized organizations**
- **Hybrid Architectures**

Centralized organizations

Simple client-server architecture:

- Processes in a distributed system are divided into two groups.
- A **server** is a process implementing a specific service

Example: a file system service or a database service.

- A **client** is a process that requests a service from a server by sending it a request and subsequently waiting for the server's reply.
- This client-server interaction, also known as **request-reply behavior**.

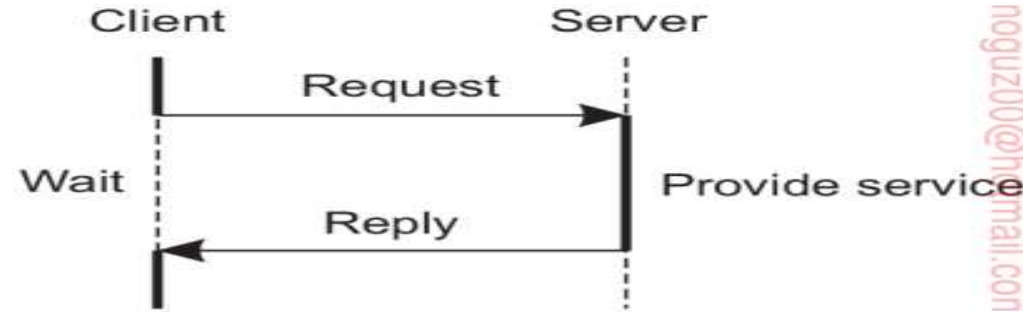


Figure 2.15: General interaction between a client and a server.

Communication between a client and a server can be implemented by means of a simple **connectionless protocol**.

- When a client requests a service, it simply packages a message for the server, identifying the service it wants, along with the necessary input data.
- The message is then sent to the server.
- Server process it, and package the results in a reply message that is then sent to the client.

- As long as messages do not get lost or corrupted, the request/reply protocol works fine.
- Unfortunately, making the protocol resistant to occasional transmission failures is not trivial.
- Client resend the request when no reply message comes in.
- But, client cannot detect whether the original request message was lost, or that transmission of the reply failed.
- If the reply was lost, then resending a request may result in performing the operation twice.
- When an operation can be repeated multiple times without harm, it is said to be idempotent.

- As an alternative, many client-server systems use **a reliable connection oriented protocol.**
- whenever a client requests a service, it first sets up a connection to the server before sending the request.
- The server uses that same connection to send the reply message, after which the connection is torn down.

Problems:

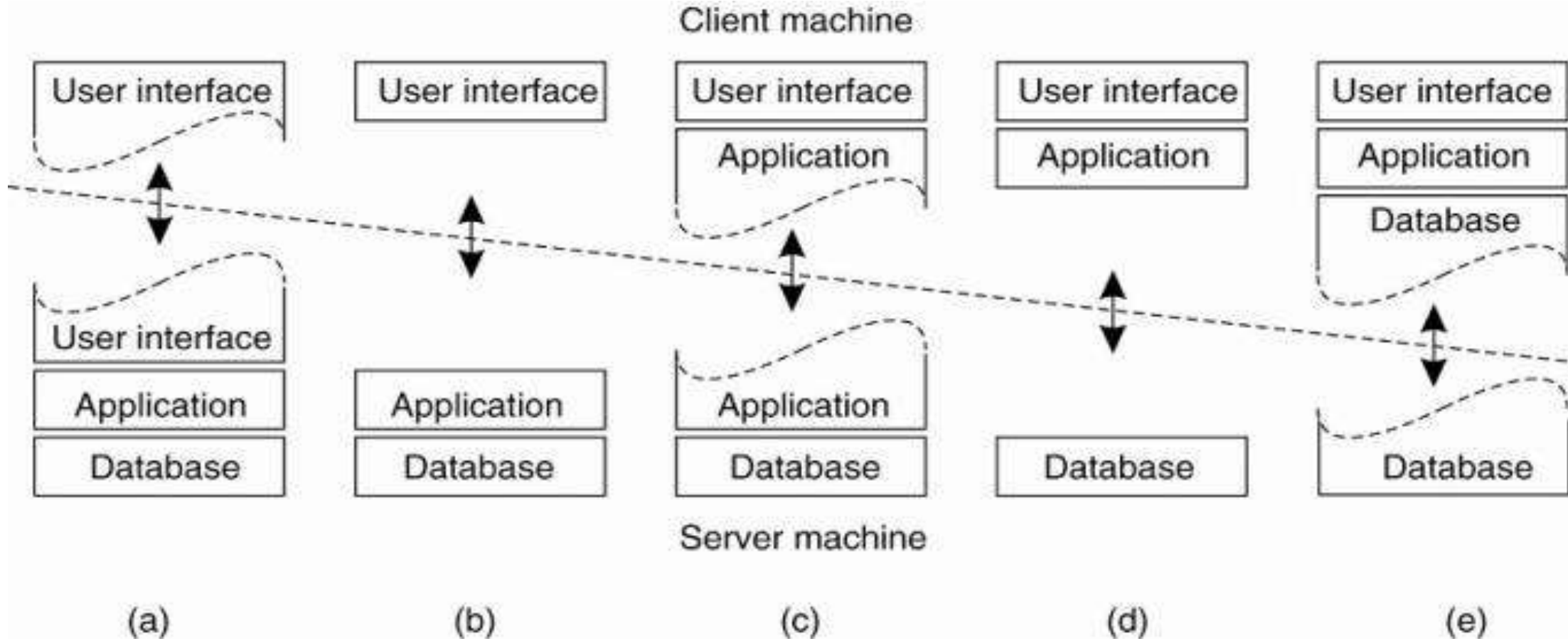
- Setting up and tearing down a connection is relatively costly, especially when the request and reply messages are small.
- No clear distinction between a client and a server.

Example:

- A server for a distributed database may continuously act as a client because it is forwarding requests to different file servers responsible for implementing the database tables.
- In such a case, the database server itself only processes the queries.

Multi-tiered Architectures

- **Two types of machines:**
 - A client machine containing only the programs implementing the user-interface level.
 - A server machine containing the rest, that is, the programs implementing the processing and data level.
- Distributed applications are divided into the three layers:
 - (1) User interface layer
 - (2) Processing layer
 - (3) Data layer
- Two kinds of machines:
- Client machines and server machines → referred to as a **Two-tiered architecture**.



Client-server organizations in a two-tiered architecture.

Cases:

A: only the terminal-dependent part of the user interface on the client machine

B: Place the entire user-interface software on the client side

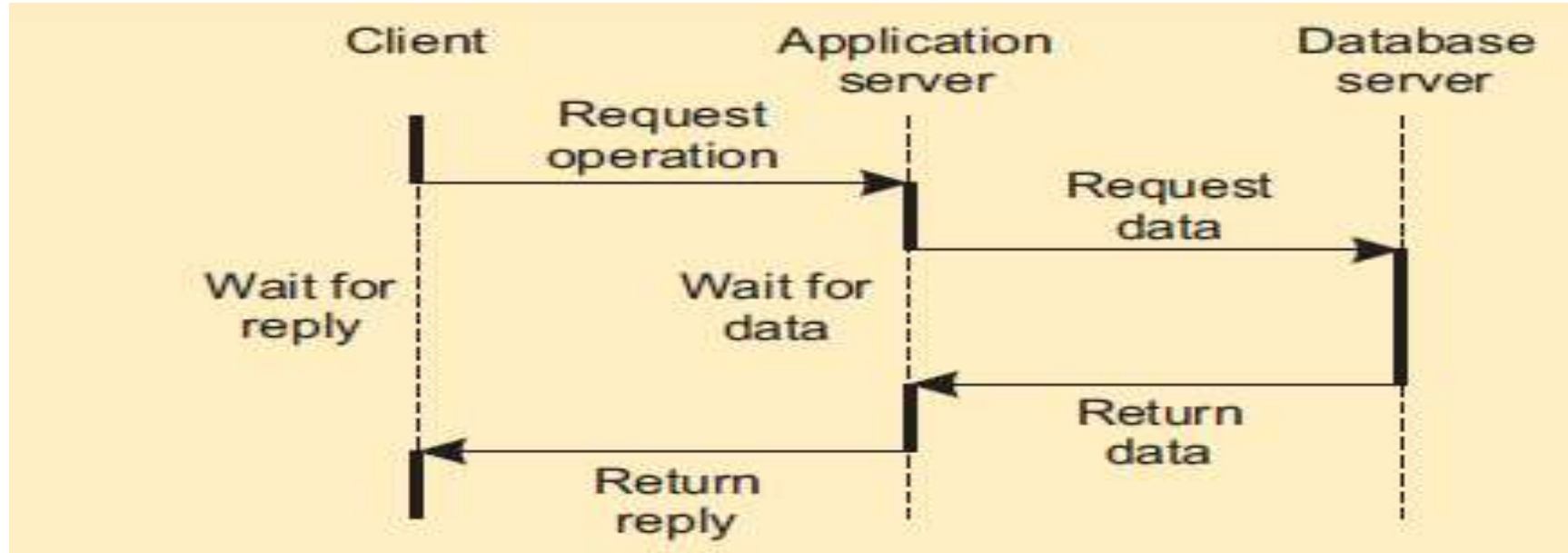
- o Divide the application into a graphical front end, which communicates with the rest of the application (residing at the server) through an application-specific protocol.
- o the front end (the client software) does no processing other than necessary for presenting the application's interface

C: Move part of the application to the front end

- o e.g. the application makes use of a form that needs to be filled in entirely before it can be processed
- o front end can then check the correctness and consistency of the form, and where necessary interact with the user

- D.** Used where the client machine is a PC or workstation, connected through a network to a distributed file system or database
- o most of the application is running on the client machine, but all operations on files or database entries go to the server
 - o e.g. many banking applications run on an end-user's machine where the user prepares transactions and such once finished, the application contacts the database on the bank's server and uploads the transactions for further processing
- E:** Used where the client machine is a PC or workstation, connected through a network to a distributed file system or database the situation where the client's local disk contains part of the data

- **Three-tier architecture**



An example of a server acting as client.

- Server may sometimes need to act as a client
- In this architecture, part of the processing layer are executed by a separate server, but may additionally be partly distributed across the client and server machines
- Ex:TPM, Website Organization

Decentralized organizations

- Distributed processing is equivalent to organizing a client-server application as a multi-tiered architecture.

Vertical distribution:

Achieved by placing logically different components on different machines.

- Term is related to the concept of **vertical fragmentation** as used in distributed relational databases, where it means that tables are **split column-wise**, and subsequently distributed across multiple machines

Horizontal distribution :

- A client or server may be physically split up into logically equivalent parts, but each part is operating on its own share of the complete data set.

e.g **peer-to-peer systems**.

- Processes are all equal: the functions that need to be carried out are represented by every process → each process will act as a client and a server at the same time (i.e., acting as a servant)

Structured Peer-to-Peer Architectures:

The P2P overlay network consists of all the participating peers as network nodes.

- There are links between any two nodes that know each other:
i.e. if a participating peer knows the location of another peer in the P2P network, then there is a directed edge from the former node to the latter in the overlay network.
- Based on how the nodes in the overlay network are linked to each other, we can classify the P2P networks as **unstructured or structured**.
- Some well known **structured P2P networks** are [Chord](#), [Pastry](#), [Tapestry](#), CAN, and [Tulip](#)

Structured peer-to-peer systems

- In a structured peer-to-peer system, the nodes are organized in an overlay that adheres to a specific, deterministic topology: a ring, a binary tree, a grid, etc
 - This topology is used to efficiently look up data.
 - Characteristic for structured peer-to-peer systems:
 - based on semantic-free index i.e., each data item that is to be maintained by the system, is uniquely associated with a key
$$\text{key}(\text{data item}) = \text{hash}(\text{data item's value}).$$
 - The peer-to-peer system as a whole is now responsible for storing (key,value) pairs.
- Most-used procedure - organize the processes through a Distributed Hash Table (DHT).

- Now, structured peer-to peer systems being able to look up a data item by means of its key.
- i.e., the system provides an efficient implementation of a function lookup that maps a key to an existing node:

existing node = lookup(key).

Example1: Simple peer-to-peer systems with a fixed number of nodes

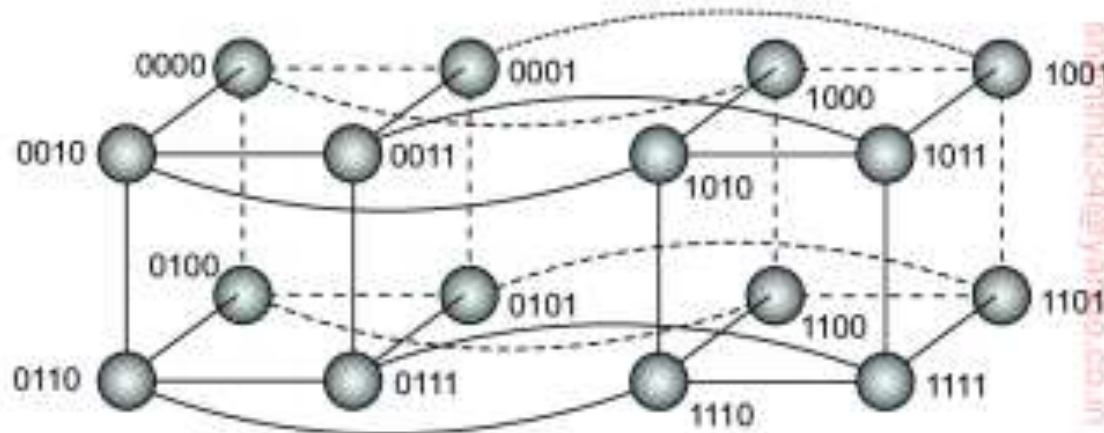


Figure 2.18: A simple peer-to-peer system organized as a four-dimensional hypercube.

- **Example 2: Chord system:** the nodes are logically organized in a ring such that an item with key k is mapped to the node with the smallest identifier id_k .
- This node is referred to as the successor of key k and denoted as $\text{succ}(k)$
- To look up the data item an application running on an arbitrary node would then call the function $\text{LOOKUP}(k)$ which would subsequently return the network address of $\text{succ}(k)$.
- At that point, the application can contact the node to obtain a copy of the data item.
- The mapping of data items onto nodes in Chord.

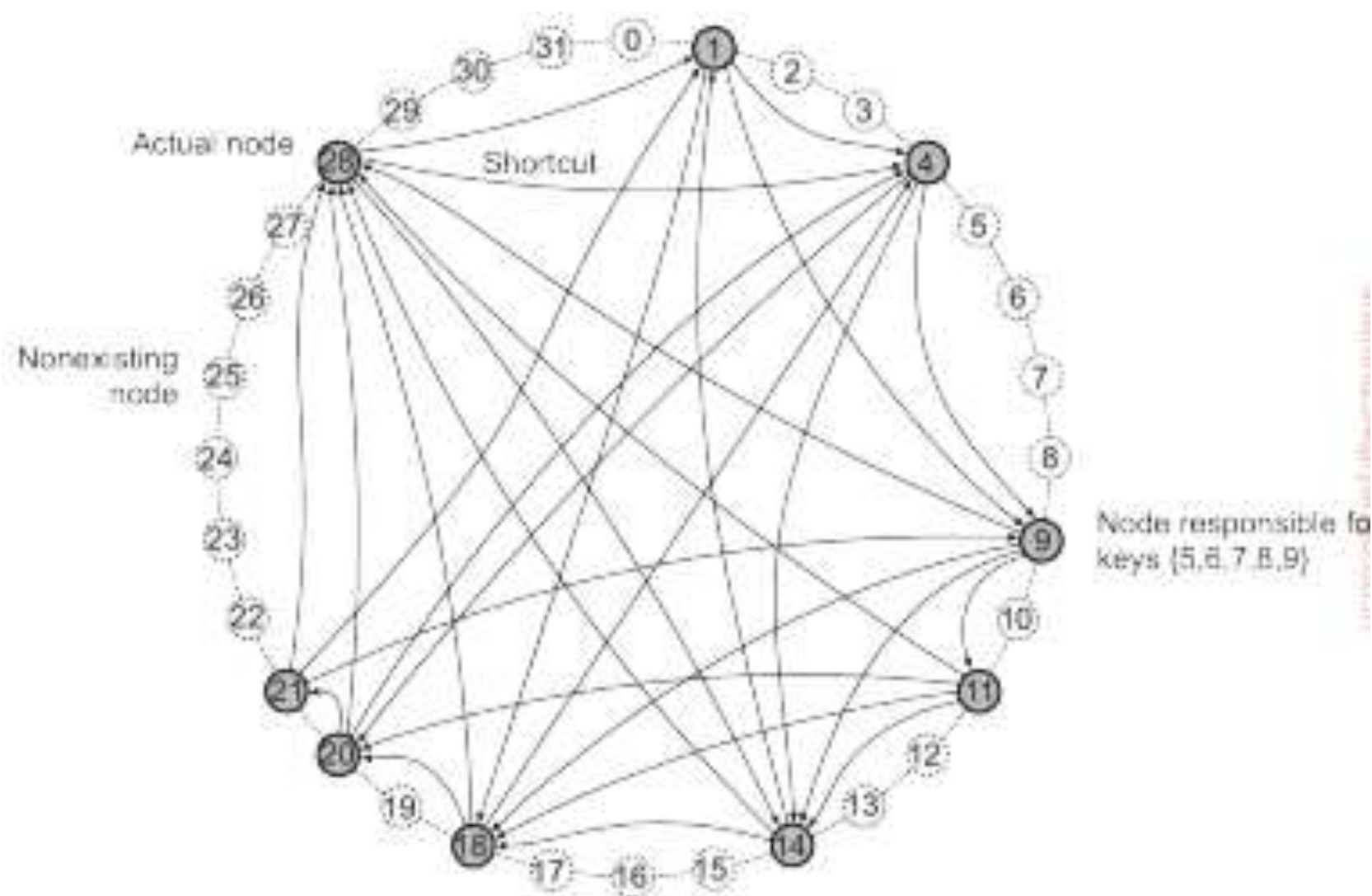


Figure 2.19: The organization of nodes and data items in Chord.

- Looking up a key does not follow the logical organization of nodes in the ring.
- Each node will maintain shortcuts to other nodes → lookups can generally be done in $O(\log(N))$ number of steps, where N is the number of nodes participating in the overlay

Joining the P2P Network

1. When a node wants to join the system, it starts with generating a random identifier id.
2. Then, the node can simply do a lookup on id, which will return the network address of $\text{succ}(\text{id})$
3. The joining node then contacts $\text{succ}(\text{id})$ and its predecessor and insert itself in the ring.
 - a. This scheme requires that each node also stores information on its predecessor.
 - b. Insertion also yields that each data item whose key is now associated with node id, is transferred from $\text{succ}(\text{id})$.

Leaving the P2P Network

1. node id informs its departure to its predecessor and successor
2. transfers its data items to succ(id).

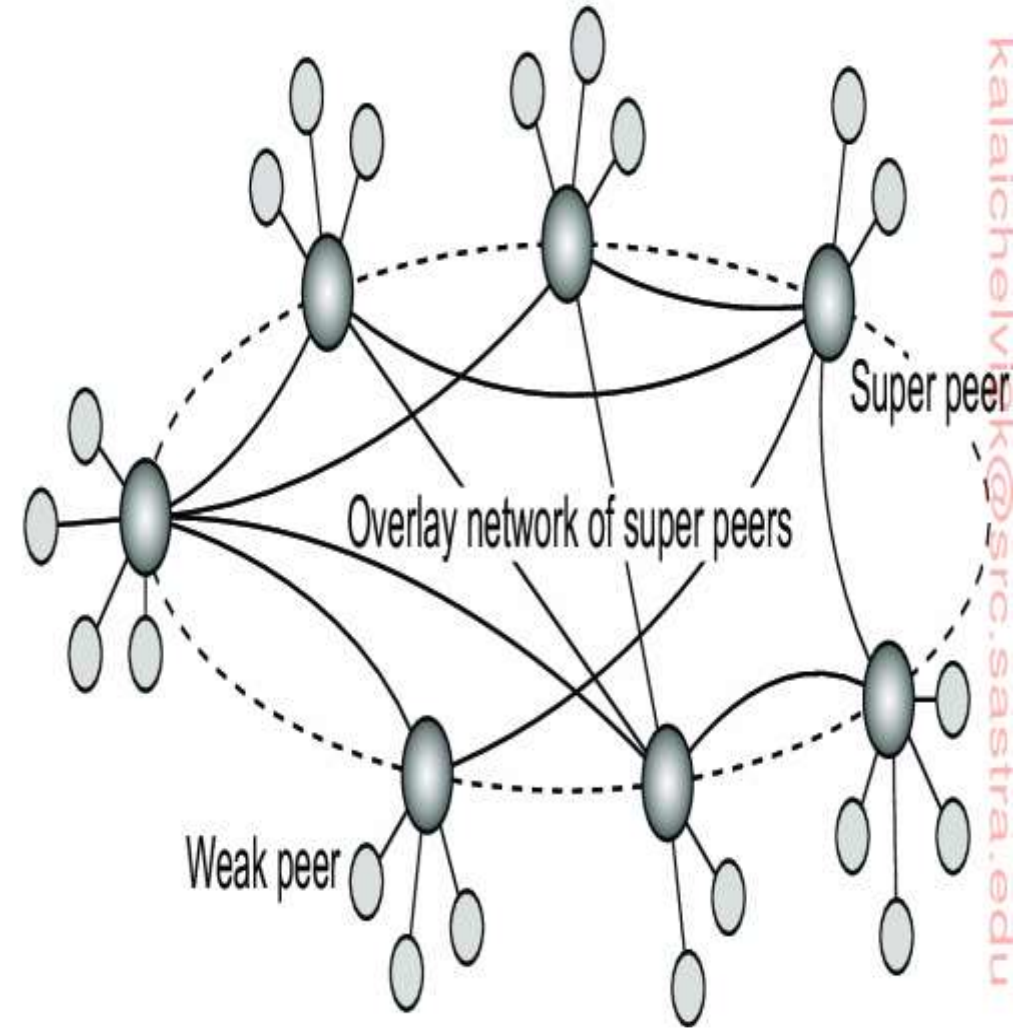
Unstructured Peer-to-Peer Architectures

- An unstructured P2P network is formed when the overlay links are established arbitrarily.
- Such networks can be easily constructed as a new peer that wants to join the network can copy existing links of another node and then form its own links over time.
- In an unstructured P2P network, if a peer wants to find a desired piece of data in the network, the query has to be **flooded** through the network in order to find as many peers as possible that share the data.
- Main disadvantage - queries may not always be resolved.
- Flooding also causes a **high amount of signaling traffic in the network** and hence such networks typically have **very poor search efficiency**.

- At the other end of the search spectrum, an issuing node u can simply try to find a data item by asking a randomly chosen neighbor, say v .
- If v does not have the data, it forwards the request to one of its randomly chosen neighbors, and so on.
- The result is known as a **random walk**
 - **less network traffic**

Hierarchically organized peer-to-peer network

- Consider a collaboration of nodes that offer resources to each other.
- **Example:**
- In a collaborative **content delivery network (CDN)**, nodes may offer storage for hosting copies of Web documents allowing Web clients to access pages nearby, and thus to access them quickly.
- In that case, making use of a **broker** that collects data on resource usage and availability for a number of nodes that are in each other's proximity will allow to quickly select a node with sufficient resources.
- Those maintaining an index or acting as a broker are generally referred to as **super peers**.



- In this organization, every regular peer, now referred to as a **weak peer**, is connected as a client to a super peer.
- All communication from and to a weak peer proceeds through that peer's associated super peer.
- The client-super peer relation is fixed in many cases.
- whenever a regular peer joins the network, it attaches to one of the super peers and remains attached until it leaves the network.
- Expected that super peers are long-lived processes with a high availability.
- To compensate for potential unstable behavior of a super peer, backup schemes can be deployed, such as pairing every super peer with another one and requiring clients to attach to both.

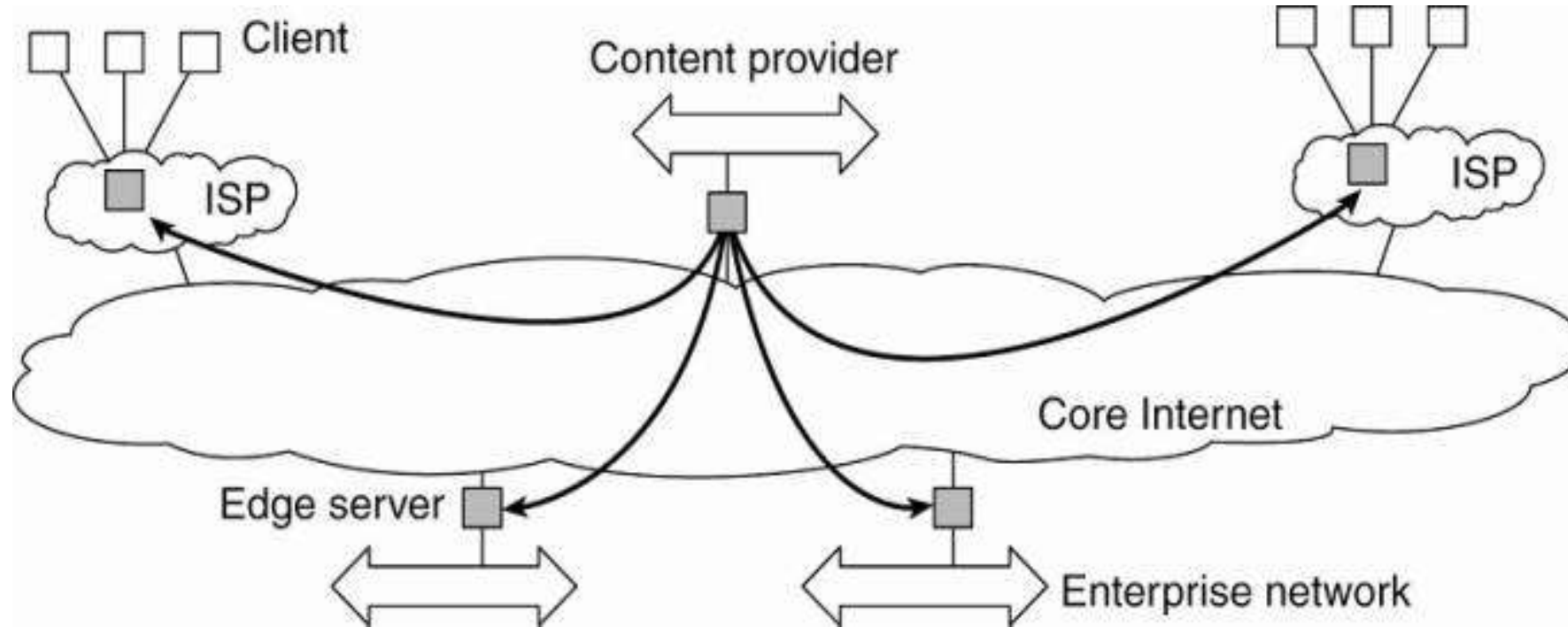
Hybrid Architectures

- Distributed systems in which client-server solutions are combined with decentralized architectures.

Edge-Server Systems

- Deployed on the Internet where servers are placed "**at the edge**" of the network.
- Purpose is to serve content, possibly after applying filtering and transcoding functions
- a collection of edge servers can be used to optimize content and application distribution
- This edge is formed by the boundary between enterprise networks and the actual Internet

- e.g, an Internet Service Provider (ISP).
- e.g. end users at home connect to the Internet through their ISP, the ISP can be considered as residing at the edge of the Internet.



Viewing the Internet as consisting of a collection of edge servers.

- one edge server acts as an origin server from which all content originates.
- That server can use other edge servers for replicating Web pages

Collaborative Distributed Systems

- Hybrid architectures are deployed in collaborative distributed systems.

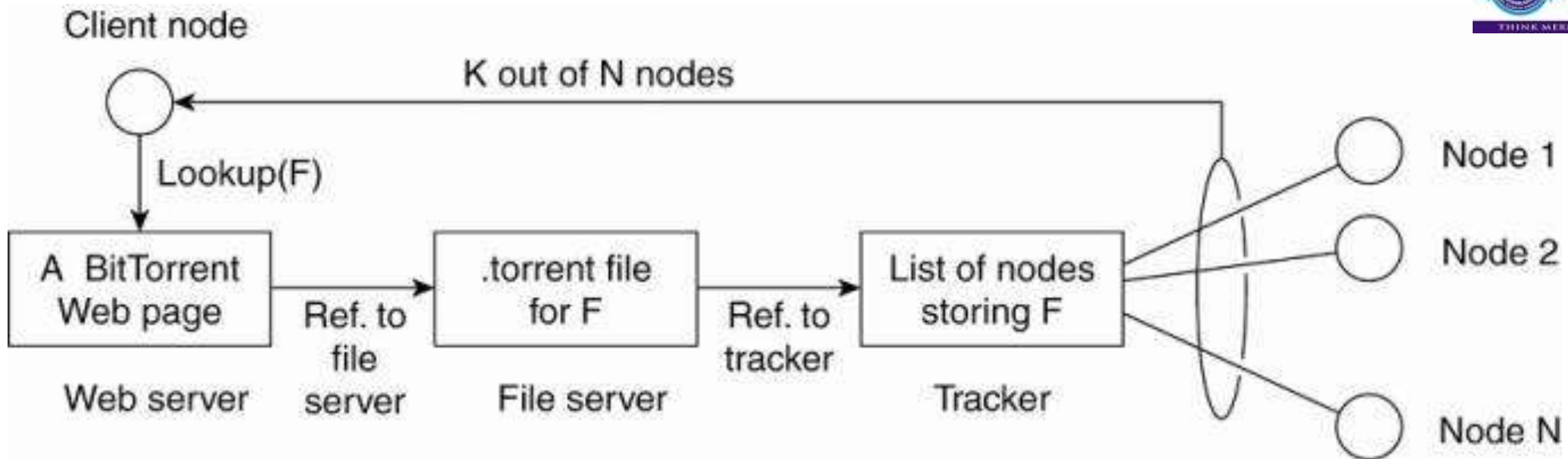
Two step process:

1. Join system using a traditional client-server scheme.
2. Once a node has joined the system - use a fully decentralized scheme for collaboration.

Example:

BitTorrent file-sharing system

- BitTorrent is a peer-to-peer file downloading system.
- An end user downloads chunks of a file from other users until the downloaded chunks can be assembled together yielding the complete file.
- BitTorrent combines centralized with decentralized solutions.



The principal working of BitTorrent

File download process:

1. Access a global directory of one of a few well-known Web sites.
 - Directory contains references to what are called .torrent files.
 - A torrent file contains the information that is needed to download a specific file.
 - It refers to a tracker - a server that keeps an accurate account of active nodes that have (chunks) of the requested file.
 - An active node is one that is currently downloading another file.
 - Many different trackers - but only a single tracker per file (or collection of files).

2. Once the nodes have been identified from where chunks can be downloaded - the downloading node becomes active.

- This node will be forced to help others by providing chunks of the file it is downloading that others do not yet have.
- Enforcement comes from a very simple rule: if node P notices that node Q is downloading more than it is uploading, P can decide to decrease the rate at which it sends data to Q.
- This scheme works well provided P has something to download from Q.
- For this reason, nodes are often supplied with references to many other nodes putting them in a better position to trade data.